

Unified Candle Intake

A coalescing, pipelined drain with one code path for every tier

This document is the design basis for the TT-164 drain phase. It states the measured problem in the candle intake path, the design invariant chosen to fix it, the drain mechanism, and the exact code locations where the changes apply. The setting: the `subscribe` service consumes vendor market data over one DXLink websocket and republishes it to Redis; “tier” refers to the fast/slow candle-channel split of TT-164, summarized in section 1.1 — no other background is assumed.

Contents

- 1 The Measured Problem**
 - 1.1 Two incidents, one drain
 - 1.2 The cost of one event
- 2 The Design Invariant**
- 3 The Coalescing, Pipelined Drain**
- 4 Where the Changes Apply**
 - 4.1 The listener loop — `handlers.py`
 - 4.2 The Redis processor — `processors/redis.py`
 - 4.3 The subscription-status write
 - 4.4 What does not change
- 5 Deferred: Channel Granularity**
- 6 Verification**

Audience and Scope

Written for the project owner and any future implementer of the TT-164 drain phase. Every code excerpt was read from the working tree on 2026-09-01; every measurement is from the live incidents of 2026-08-31 and 2026-09-01, taken during those sessions. This is a design document: the “revised” listings in section 4 are sketches to be refined at implementation time, not merged code.

1 The Measured Problem

The candle intake drains events more slowly than the market produces them at the open. The deficit accumulates in unbounded in-process queues, so consumers of the affected channel read bars that are minutes to hours old while the process itself looks healthy. Two production incidents in two days established this. The second one isolated the cause.

Two terms recur. A **DXLink channel** is a numbered logical stream inside the one vendor websocket; each channel has its own subscriptions, its own server-side aggregation period, and — in our consumer — its own `asyncio.Queue` and drain coroutine. A candle bar is **sealed** when the next bar begins; until then the vendor re-publishes the same bar with updated values on every tick, and the signal engine acts only on sealed bars.

1.1 Two incidents, one drain

2026-08-31. From the opening volume ramp, every candle subscription on the single shared channel fell behind together — about 55 minutes at the worst. The connection never dropped, so no reconnect fired. The TT-156 collector — the paper-trading process that consumes SPX bars from Redis — saw no bar seal for almost two hours. Its signal engine was fed one stale bar mutating in place and then a compressed catch-up burst, and it produced a *wrong-direction* trade, not merely a late one. The same-day response was TT-164: candle subscriptions split across two DXLink channels — a configured fast pool (SPX 1m/5m) at 0.1 s server-side aggregation, everything else conflated to 1.0 s.

2026-09-01. The split held: the fast pool stayed at zero lag all session and the collector traded on time. But the slow channel fell roughly two hours behind and stayed there through midday. That divergence is the diagnostic fact: both channels share the same websocket, so a wire or vendor problem would delay both. Only the consumer side distinguishes them.

Measurement (2026-09-01, 12:22 ET)	Value
Fast channel (SPX 1m/5m, 2 subscriptions)	Current — the 12:20 ET bar delivered at 12:21 ET
Slow channel (34 subscriptions)	~2 h behind — AAPL 1m delivering the 10:19 ET bar at 12:22 ET
Publish-lag warnings since the open	195, ramping from 09:xx, peaking 10:00–12:00 (the Redis publisher warns when a candle's bar

Measurement (2026-09-01, 12:22 ET)	Value
	time trails wall clock; <code>warn_if_stale</code> , section 4.2)
Subscribe process CPU	40% of one core — not saturated
Subscribe process memory	1.22 GB, up from 0.50 GB the prior midday

The memory growth is the queued backlog itself: the per-channel `asyncio.Queue` is unbounded by design — a deliberate earlier decision, recorded in `sockets.py` against TT-161 — so two hours of undelivered events sit in RAM. Forty percent CPU with a growing queue means the drain is not compute-bound; it is waiting. Section 1.2 shows what it waits on.

1.2 The cost of one event

Each channel is drained by one coroutine that processes strictly one event at a time. For every event, that coroutine awaits **four Redis round-trips in sequence** before it takes the next event:

Await, in order	Where
1. Pub/sub publish	<code>RedisEventProcessor.process_event</code> — <code>processors/redis.py:60</code>
2. Latest-value <code>HSET</code>	same method — <code>processors/redis.py:64</code>
3. Status <code>HGET</code>	<code>RedisSubscriptionStore.update_subscription_status</code> — <code>connections/subscription.py:119</code>
4. Status <code>HSET</code>	same method — <code>connections/subscription.py:123</code>

Per-event Pydantic model construction and JSON serialization add to that unit cost, and the slow channel's 34 subscriptions all pass through it serially. At the open, arrival outruns the drain. The queue absorbs the difference, and the channel spends the rest of the day working off the backlog. The fast channel runs the identical code and stays current only because two subscriptions produce so much less work.

WHY INFLUXDB IS NOT THE BOTTLENECK

The InfluxDB processor already uses the approach this document proposes for Redis: `process_event` only appends the point to a buffer, and a dedicated writer thread flushes batches (`processors/influxdb.py`; batching on a writer thread landed under TT-108, sealed-bar-only writes under TT-159). No InfluxDB call blocks the event loop, `influxd` sat at 5% CPU during both incidents, and no write errors were logged.

2 The Design Invariant

The fix keeps **one code path for every candle channel**. A DXLink channel is configured by exactly two things: which subscriptions it carries and its aggregation period. Nothing else about a channel varies. “Fast” and “slow” are configuration values, not kinds of channel, and no branch in the intake code tests which tier it is on.

One code path. Latency determined by load, throughput by backlog, configuration by the pool.

– design invariant, TT-164 drain phase

Spelled out: a current channel delivers each event at per-event latency; a backlogged channel batches, and the batch size follows the backlog depth; which subscriptions run at full rate is decided solely by the `CANDLE_FAST_POOL` list — a configured list of individual subscriptions such as `SPX{=5m}` — and by nothing in code.

The invariant is achievable because the drain mechanism in section 3 behaves correctly at both extremes without modes. When a channel’s queue is empty, the mechanism publishes each event immediately — identical to today’s behavior. When a queue holds a backlog, the same mechanism condenses the backlog into few round-trips — the only case where condensing is wanted. A channel that is keeping up cannot tell the new drain from the old one.

Three prior decisions stand and are inputs here, not open questions:

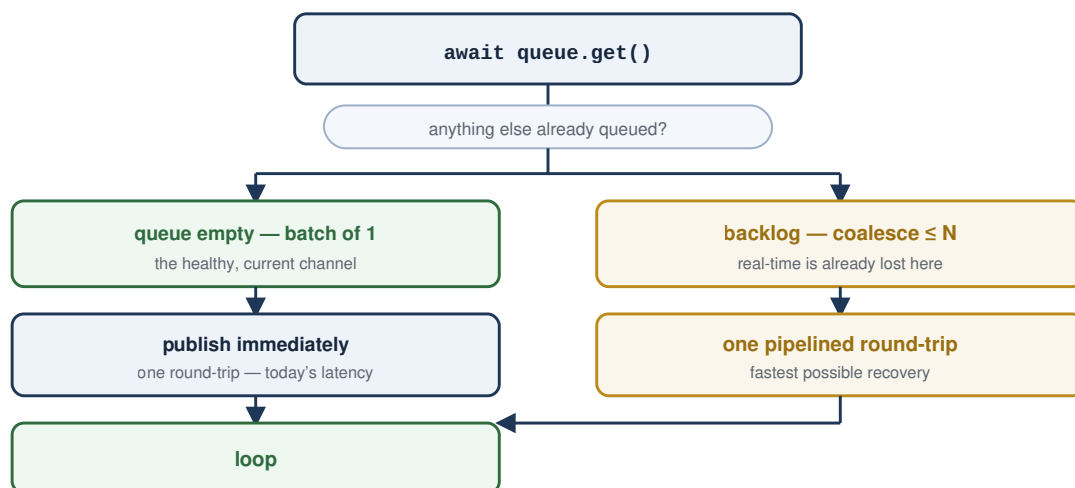
- **TT-164 stays as shipped** — the fast-pool list (`CANDLE_FAST_POOL`) and per-channel aggregation periods are the configuration surface this design reads.
- **The real-time path is never slowed** — no timers, no minimum batch sizes, nothing that delays an event on a current channel (owner requirement, 2026-09-01 discussion).
- **Drain first, channels later** — coalescing and pipelining are implemented on today’s two channels before any further channel split is considered (owner sequencing decision; section 5).

3 The Coalescing, Pipelined Drain

The drain loop changes from “take one event, finish it, repeat” to “take everything already queued, process it as one batch, repeat.” Two operations make the batch cheap:

- **Coalescing** — the listener empties the queue in one pass: one blocking `await queue.get()` for the first event, then `get_nowait()` until the queue is empty (with a size cap so a deep backlog is worked in slices).
- **Pipelining** — the batch’s Redis commands are sent in one round-trip using a Redis pipeline, instead of four awaited round-trips per event.

The degenerate case is the important one. An empty queue yields a batch of one, and a batch of one is published immediately. The pipeline then holds two commands (the publish and the latest-value write) and costs one round-trip where those two cost two today; the other two round-trips of section 1.2 — the status stamp — are removed separately (section 4.3). A current channel therefore keeps per-event delivery latency. A backlogged channel has already lost real-time delivery; the batched drain recovers it at the maximum possible rate. The fast pool will form batches larger than one during opening bursts; that is the mechanism absorbing the burst in one round-trip rather than queueing it.



The drain loop. Both branches are the same code; the batch size is decided by what the queue already holds, never by a timer.

4 Where the Changes Apply

Three code locations change; two explicitly do not. Each subsection shows the current code (read from the working tree, 2026-09-01) and the revised shape as a design sketch.

4.1 The listener loop — `src/tastytrade/messaging/handlers.py`

`EventHandler.queue_listener` is the serial drain: one event per iteration, fully processed before the next.

```
# current — handlers.py, EventHandler.queue_listener
while not self.stop_listener.is_set():
    reply = await queue.get()
    self.metrics.update(queue.qsize())
    message = Message(...)
    # 4 awaited Redis round-trips inside:
    await self.handle_message(message)
    queue.task_done()
```

Revised: coalesce everything already queued (bounded by a slice cap), parse all of it, then pass the whole batch to the processors once.

```
# sketch — same method, coalescing drain
DRAIN_SLICE = 500    # work a backlog in bounded slices

while not self.stop_listener.is_set():
    replies = [await queue.get()]
    while len(replies) < DRAIN_SLICE:
        try:
            replies.append(queue.get_nowait())
        except asyncio.QueueEmpty:
            break
    self.metrics.update(queue.qsize())
    # parse_message = the Message(...) construction and
    # event decoding lifted out of today's handle_message
    events = [e for r in replies
               for e in self.parse_message(r)]
    # one processor call for the whole batch:
    await self.dispatch_batch(events)
```



```
for _ in replies:
    queue.task_done()
```

The per-event processor fan-out inside `handle_message` moves with it. Today every event is offered to every processor individually:

```
# current – handlers.py:208-215 (reflowed to fit)
for event in events:
    for _, processor in self.processors.items():
        result = processor.process_event(event)
        if asyncio.iscoroutine(result):
            await result
    if self.subscription_store and \
        hasattr(event, "eventSymbol"):
        await self.subscription_store \
            .update_subscription_status(
                event.eventSymbol, {})
```

```
# sketch – dispatch_batch
async def dispatch_batch(self, events):
    for _, processor in self.processors.items():
        # batch-aware; default loops process_event
        result = processor.process_events(events)
        if asyncio.iscoroutine(result):
            await result
    # throttled status stamps – section 4.3:
    await self.touch_subscriptions(events)
```

`process_events` gets a default implementation on `BaseEventProcessor` that loops over `process_event`. Every existing processor therefore works unchanged: the candle sealer (holds each forming bar, forwards it once sealed), the backfill snapshot tracker (marks a subscription live after its first snapshot), and the InfluxDB buffer of section 1.2. Only the Redis processor overrides the batch method.

4.2 The Redis processor — `src/tastytrade/messaging/processors/redis.py`

Today's per-event method awaits two round-trips:

```
# current — RedisEventProcessor.process_event
event_json = event.model_dump_json()
...
await self.redis.publish(channel=channel, message=event_json)
await self.redis.hset(hset_key, symbol, event_json)
```

The override serializes each event once and sends the whole batch's commands in a single pipelined round-trip. The per-event lag check (`warn_if_stale`) is kept, per event, before the send.

```
# sketch — RedisEventProcessor.process_events
async def process_events(self, events):
    pipe = self.redis.pipeline(transaction=False)
    for event in events:
        event_json = event.model_dump_json()
        event_type = event.__class__.__name__
        symbol = event.eventSymbol
        if event_type == "CandleEvent":
            self.warn_if_stale(event)
        pipe.publish(
            f"market:{event_type}:{symbol}", event_json)
        pipe.hset(
            f"tastytrade:latest:{event_type}",
            symbol, event_json)
    # one round-trip for the whole batch:
    await pipe.execute()
```

Ordering. A non-transactional pipeline preserves command order on one connection, so subscribers still see events in arrival order, and the latest-value hash is written after its event's publish exactly as today.

4.3 The subscription-status write — `src/tastytrade/connections/subscription.py`

Half of the four round-trips are bookkeeping. `update_subscription_status` does an `HGET`, a JSON decode/encode, and an `HSET` to stamp `last_update` — today per event, per symbol.

```
# current – update_subscription_status (reflowed)
if data_bytes := await self.redis.hget(
    self.hash_key, symbol):
    sub = json.loads(data_bytes.decode("utf-8"))
    sub["last_update"] = \
        datetime.now(timezone.utc).isoformat()
    sub["metadata"].update(data)
    await self.redis.hset(
        self.hash_key, symbol, json.dumps(sub))
```

The batch dispatch stamps each distinct symbol in the batch at most once, and skips a symbol stamped within the last second. This loses nothing, because the stamp has exactly two readers, both coarse: after a disconnect, the reconnect path takes the oldest `last_update` across subscriptions as the date to backfill from (`subscription_orchestrator.py`, `get_reconnect_start`), and the health display shows each subscription's last-seen time. Both work at second granularity or coarser. Under backlog the throttle collapses thousands of round-trips into a few; on a current channel it is one stamp per event, as today, minus the redundant re-stamps within the same second.

4.4 What does not change

Unchanged	Why
<code>processors/influxdb.py</code>	Already buffered and flushed in batches on a dedicated writer thread (TT-108); already writes only sealed bars (TT-159).
TT-164 channels and pool config	The three <code>CANDLE_*</code> settings are the configuration surface; the drain reads nothing tier-specific.
Channel count	Two candle channels, until the metrics say otherwise (section 5).
Candle sealing, snapshot tracking	They receive the same events via the default <code>process_events</code> loop.

5 Deferred: Channel Granularity

Per-subscription channels — every subscription its own DXLink channel with its own aggregation period — remain the end-state candidate. The owner’s sequencing decision (2026-09-01) defers them. The drain fix addresses throughput, the deficit both incidents demonstrated. More channels address isolation between subscriptions, and that value shrinks once the drain outruns arrival. The refactor is also the larger one: the static `Channels` enum (`config/enumerations.py`), `CHANNEL_SPECS` (`config/configurations.py`), the queue dictionary, the router handler map, and reconnect restore all assume a fixed channel set today.

Option	Gains	Cost
A — two channels + this drain	Throughput; burst absorption	Small diff in three files — this document
B — channel per symbol (6 today)	A flooded symbol only delays itself	Moderate; the <code>Channels</code> enum stays a fixed list
C — channel per subscription (36)	Full isolation; per-subscription aggregation as a channel property	Dynamic channel registry; the largest refactor

The decision gate is empirical: implement A, then watch the queue-depth and publish-lag metrics across several market opens. If any channel still accumulates a backlog it cannot drain within minutes, B/C get their design discussion with that data as input. If not, they stay deferred.

OPEN · DEPENDS ON TT-158

The gate needs continuous queue-depth metrics, which do not exist yet — today `QueueMetrics` values are logged only at shutdown. TT-158 (pipeline health observability) supplies them and should land with or before this change so the before/after is measured, not anecdotal.

6 Verification

6.1 Unit tests

- **Batch-of-one identity** — with an empty queue, the drain publishes one event immediately; Redis receives publish + hset for it in order.
- **Coalescing** — with $K \leq$ slice-cap events queued, one listener iteration consumes all K and issues one `execute()`.
- **Slice cap** — with more than the cap queued, the batch stops at the cap and the next iteration continues; no event is lost or reordered.
- **Status throttle** — a symbol appearing many times in one batch is stamped once; a symbol stamped under one second ago is skipped.
- **Processor compatibility** — a processor implementing only `process_event` receives every event of a batch, in order, via the default `process_events`.

6.2 Live evidence (PR requirement)

- A market open on the slow channel with **zero** sustained publish-lag warnings, against 195 on 2026-09-01.
- Queue depth (TT-158) returning to zero within seconds of the opening burst on every channel.
- Flat process memory across the session, against $0.5 \rightarrow 1.2$ GB.
- Unchanged SPX 1m/5m delivery latency on the fast pool — the collector seals every bar on time, as it already does.